Fault-tolerance in Ant Colony Optimization Network Routing

Zihan Pan, Shuo Zhang, Kohtaro Uchida

Abstract

Ant Colony Optimization learns through repeated path sampling and pheromone based reinforcement. This optimization process can be applied in graph-based problems, including network routing. While AntNet-style probabilistic routing can adapt to congestion and topology changes, wireless multi-hop environments introduce frequent topology changes and signal interferences, which can leave stale pheromone entries and slow recovery.

In order to investigate the trade-off between fault-tolerance and control overhead, we implement an AntNet routing protocol in ns-3 by extending `Ipv4RoutingProtocol` and integrate two fault-tolerance mechanisms inspired by prior ACO-based routing literature: (i) beacon-based neighbour monitoring with directional pheromone evaporation, and (ii) link failure notification propagation. We describe the protocol design and ns-3 integration details. We also measure and analysis the effectiveness and overhead of the two fault-tolerance add-on mechanisms in comparison with vanilla AntNet.

1 Introduction

Network Routing is an online optimization problem. Ant Colony Optimization (ACO) [1] offers an appealing algorithmic template for this setting. By combining stochastic exploration with pheromone-based reinforcement, ACO enables distributed ant agents to collectively learn good solutions without centralized control or complete global knowledge. AntNet [2] is a representative ant-inspired routing protocol that realizes the ACO principle through probabilistic routing tables updated by forward and backward ant agents. Continuous path sampling on good routes, probabilistic next-hop selection, and multi-path availability makes AntNet attractive for dynamic, distributed, and resource constrained environments.

Wireless multi-hop networks, however, are harsher than the abstract graphs typically assumed by classical optimization. Compared with traditional wired networks, node mobility, interference, and contention on shared medium can cause links to be asymmetric, transient, or intermittently unavailable [3][4], which distort delay measurements, invalidate previously reinforced next-hop preferences, and may leave learned routing tables "stale." As a result, a routing protocol should not only learn good paths, but also actively detect and recover when underlying topology changes.

This motivates the central question of this project: how do explicit fault-handling mechanisms behave in an ACO-style routing protocol under wireless dynamics? In this project, we implement ACO-style probabilistic routing protocol in ns-3 by extending `Ipv4RoutingProtocol` class [5], and we investigate two complementary fault-handling mechanisms adapted from prior ant-based routing designs. The first mechanism is beacon-driven failure detection with directional pheromone evaporation [6], which allows nodes to actively detect link failures and update pheromone without waiting forward/backward ant's exploration round trip. The second mechanism is failure notification propagation [7], which enables upstream nodes to react faster than waiting for gradual pheromone

decay. Together, these mechanisms combine proactive local detection with reactive network-wide adaptation. We evaluate their benefits and overhead by measuring packet loss ratio and end-to-end delay under different assumptions.

2 Background

2.1 Ant Colony Optimization and Network Routing

Ant Colony Optimization (ACO) [1] is a meta-heuristic inspired by the foraging behaviour of ants. In nature, ants can collectively discover the shortest route between their nest and a food source without centralized control. This process is demonstrated in Figure 1. Individual ants initially explore in different directions randomly while depositing pheromone. When an ant gets to food, it returns to the nest along the path it has taken. If the ants move in the same speed, shorter paths can be traversed more frequently within the same amount of time, and therefore tend to accumulate more pheromones. Subsequent ants are more likely to follow paths with higher pheromone concentration, creating a positive feedback loop: good paths attract more ants, and more ants further reinforce those paths. Meanwhile, pheromone evaporates through time, which prevents the colony from permanently committing to outdated choices, allowing continued exploration.

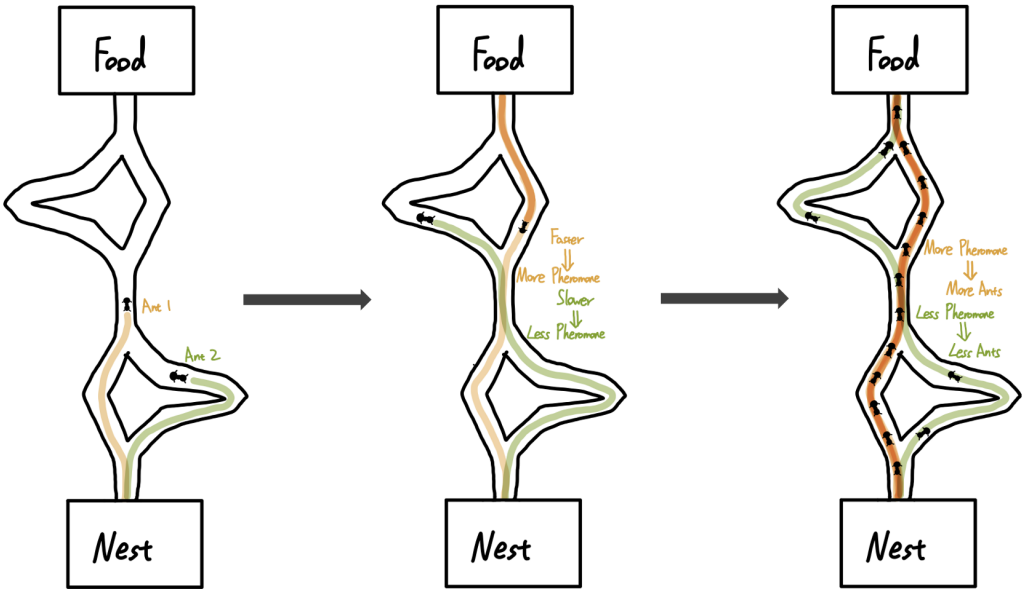


Figure 1: Ant foraging behaviour in ACO

By abstracting the nest, food sources, and intersections as nodes, and ants' movements as edges, this behaviour can be viewed as a stochastic process for searching low-cost paths in a graph. Compared with deterministic shortest-path algorithms such as Dijkstra or Bellman–Ford, ACO can be more robust to dynamic changes since it continuously samples and updates path preferences rather than committing to a single fixed route. As a meta-heuristic, however, it does not provide optimality guarantees and typically trades overheads for adaptivity.

2.2 ACO in Network Routing

Building on the ACO principle, AntNet [2] adapts this exploration process to packet-switched networks by treating the network topology as a graph and maintaining a probabilistic routing table at each node. Instead of setting a single next hop deterministically, AntNet maintains a pheromone distribution over neighbours for each destination and updates it with repeated end-to-end samples collected by control packets ("ants"). This probabilistic representation naturally supports multipath routing and enables gradual adaptation when network conditions change.

Operationally, AntNet maintains two tables as routing state at each node: a pheromone routing table and a local traffic statistics table. For each destination d, the pheromone routing table contains all the neighbours N as possible next hops, each associate with a pheromone value as a map $\langle n, p_{(d,n)} \rangle$ that represents the probability of selecting that neighbour n as the next hop to d. The local traffic statistic table stores the mean μ_d and variance σ_d^2 of sampled end-to-end delay $t_{m \to d}$ to each destination d, which is used to measure the "goodness" of observed path when updating the pheromone value with a received ant sample.

To build and refine the pheromone routing table, AntNet periodically launches *forward ant* packets that travel toward a destination in an interval of I_a, where the destination is selected based on the amount of historical data traffic. Forward ants are routed in the same manner as data packets so that they experience realistic congestion and delays. Along the way, forward ants record each visited node v and the delay t_v when it gets to v in a stack like $\langle v_1, t_{v_1} \rangle \to \langle v_2, t_{v_2} \rangle \to ... \to \langle d, t_d \rangle$.

Upon reaching the destination d, the forward ant turns into a *backward ant* and retraces the sampled path to update routing state at nodes along the path. At each node v_i on the path, based on the ant's observed delay $t_{v_i \to d} = t_d - t_{v_i}$ from the current node v_i to the destination d and historical delay from the local traffic statistics table (μ_d and σ_d^2 at node v_i), the update assigns reinforcement on the pheromone value $p_{(d,v_{i+1})}$ of the sampled next-hop choice v_{i+1}. The pheromone for the remaining neighbours are normalized so that the per-destination probabilities still sum to one. See Appendix A for the detailed math works on ant destination selection, next-hop finding, and pheromone updates.

In summary, ACO provides a distributed exploration-and-reinforcement principle, and AntNet instantiates this principle in packet-switched networks through ant-based sampling and probabilistic pheromone routing tables.

2.3 Wireless Network and Fault Tolerance

In relatively stable wired networks, AntNet's learning-driven design is well matched to the underlying dynamics. Path cost is typically affected by congestion patterns and gradual fluctuations rather than frequent topology changes. Continuously sampling end-to-end performance with ants and updating probabilistic next-hop preferences can effectively track the network state.

In contrast, wireless multi-hop network operate over a shared and unreliable medium. Link quality can vary rapidly due to interference and contention, and mobility can change the node topology on short timescales [3]. As a result, routing measurements are noisier, packet loss is more common, and links may be intermittent or asymmetric [4]. These properties make it substantially harder for a purely sample-and-reinforce mechanism to remain both responsive and stable.

These wireless characteristics expose several fault-tolerance limitations in vanilla AntNet. First, AntNet primarily updates routing state through a closed-loop sampling process: a forward ant must successfully reach the destination, and a backward ant must then return along the sampled path to apply pheromone updates. This leads to an update latency of at least one end-to-end

round trip, during which the routing table may remain biased toward stale next hops after link breaks [7]. Second, because reinforcement is driven by successful samples, the protocol has limited means to quickly produce strong negative feedback for newly broken directions, so the node may continue to favour a non-optimal next hop until sufficient successful samples accumulate on other next hop options. Third, ants may be dropped in mid-route due to packet losses, in which case the sample is lost and no update can be applied. Although increasing the forward ant generation rate may improve responsiveness under rapid environmental changes, it also introduces additional control traffic that competes with data traffic for the same channel and can increase collisions and interference, further degrading performance.

Taken together, these factors motivates augmenting AntNet-style routing with explicit fault-handling components that (i) detect neighbour/link disappearance promptly and (ii) accelerate the dissemination of failure information so that routing state can be adjusted before repeated packet losses occur. The next section describes two such mechanisms adapted from prior ant-based routing work.

3 Fault-tolerance Mechanisms

3.1 Beacon Ant and Directional Evaporation

Vanilla AntNet relies on successful forward/backward ant sampling cycles to update its routing state. In wireless multi-hop networks, this may leave stale pheromone entries after topology changes. To provide faster local reaction toward topology changes, we adopt a beacon-based monitoring and directional pheromone evaporation mechanism inspired by a fault-tolerant ACO routing design for many-to-many IoT sensor networks [6] .

In additional to forward and backward ants, each node periodically broadcast *beacon ants* to its neighbours, where beacon ant is light weight and contains only the sender's address. While broadcasting beacon ants, a node also receive beacon ants from its neighbours. We treat the beacon-sending interval I_b as a round. For each neighbour n, the node maintains a sliding window $W_n = \{w_{n,1}, w_{n,2}, ..., w_{n,K}\}$ of size K, where $w_{n,i}$ records the number of beacon ants received from neighbour n during round i. At the beginning of the next round $i+1$, the node appends the counter from round i to the window and removes the oldest entry:

$$W_n \leftarrow \{w_{n,i-K+1}, w_{n,i-K+2}, ..., w_{n,i}\} \text{ at the start of round } i+1$$

It then computes the number of consecutive zeros in the window q_n, meaning that the node has received no beacon ants from neighbour n for the most q_n rounds. A larger q_n indicates a higher likelihood that neighbour n or the link to n has broken. The evaporation factor is then calculated as:

$$\rho_n = \min(D - \frac{q_n}{w}, 1)$$

where D is a constant set as 1.35, meaning the evaporation factor would be $\rho_n = 0.35$ if the node has not received any beacon ant from neighbour n throughout the window. For each destination, the pheromone value associated with neighbour n is multiplied by ρ_n to implement this evaporation:

$$p_{(d,n)} \leftarrow \rho_n \times p_{(d,n)} \text{ for each destination } d$$

Finally, the per-destination next-hop pheromone values are re-normalized so that they sum to one. The window size K is configurable to adjust sensitivity to failures: a smaller K makes the

mechanism more aggressive, so fewer missed rounds drives ρ_n closer to the minimal value $D-1$; whereas a larger K yields a more conservative response that mitigates false positives.

In summary, beacon ants provide a lightweight, proactive signal of neighbour availability, and directional evaporation turns consecutive beacon losses into rapid local suppression of stale next-hop probabilities. This would reduce recovery latency compared with relying solely on end-to-end ant sampling. However, its effect is primarily local, which motivates a complementary mechanism that propagates failure information beyond the detecting node in the next subsection.

3.2 Messenger Ants and Failure Notification Propagation

Traditional routing protocols often incorporate some form of triggered invalidation to react quickly to link failures. For example, in distance-vector routing, techniques such as poisoned reverse [8] aim to propagate negative information so that upstream nodes do not keep forwarding traffic towards a broken direction. A similar idea also exists in AntHocNet's link failure handling [7]. In contrast, vanilla AntNet mainly relies on end-to-end sampling and reinforcement at each node, and it does not explicitly disseminate failure information. Even if a failure is detected by the added beacon ant mechanism, negative feedback is only reflected at the node that detects the failure. Without explicit failure propagation, a node that has detected a broken next hop can temporarily behave like a black hole from the perspective of upstream nodes: neighbours may continue to select it as a next hop based on stale pheromone entries, while the forwarding direction beyond it is no longer available, causing packet losses until routing preferences gradually adapt.

Inspired by this idea, we incorporate a lightweight failure notification propagation mechanism into our AntNet protocol implementation. Intuitively, if a suspected failure causes a large reduction in a node's forwarding preference for a given next hop (a large pheromone evaporation), then upstream neighbours should be informed; if the reduction is minor, local evaporation is sufficient and global propagation would only add overhead.

Suppose node m suspects that neighbour n has crashed (or the link between m and n is broken) and applies a directional evaporation toward n. Since the pheromone value associated with next hop n differs across destinations, impact of this evaporation can also differ across routing-table entries. For each destination d, let $\Delta p_{(d,n)_m}$ be the amount of pheromone reduction on entry (d,n) at node m. In the case of beacon-based pheromone evaporation from last section, this would be:

$$\Delta p_{(d,n)_m} = (1-\rho_n) \times p_{(d,n)_m}$$

where ρ_n is the evaporation factor calculated from the number of missing beacon rounds and windows size. If the pheromone reduction amount exceeds a threshold l for at least one destination, it is worthwhile to inform other neighbours. To do so, node m constructs a *messenger ant* that contains the address of the failed neighbour n together with a map of per-destination pheromone reduction amount $\{\langle d, \Delta p_{(d,n)_m}\rangle\}$. The messenger ant is broadcast to m's neighbours except the failed neighbour n.

Upon receiving a messenger ant from sender m, a neighbour j interprets the message as a warning that routes via m towards the listed destinations have become less reliable. For each destination d in the map, node j applies a proportional penalty to its own pheromone entry for choosing m as the next hop:

$$\rho_{(d,m)_j} = 1 - \Delta p_{(d,n)_m}$$

$$p_{(d,m)_j} \leftarrow \rho_{(d,m)_j} \times p_{(d,m)_j} \text{ for each entry in messenger ant}$$

Node j then evaluates whether this induced reduction $\Delta p_{(d,m)_j}$ is itself important using the same threshold rule; if so, it rebroadcasts the messenger ant to its other neighbours so that the warning propagates further upstream. Since pheromone values are bounded by 1 and the update is a reduction, the propagated effect typically decreases along the propagation chain and will eventually fall below the threshold. The threshold L is adjustable to balance propagation and overhead: a smaller L allows the failure information to reach more nodes, while a larger L stops propagation earlier to avoid broadcast storms.

This mechanism provides a reactive, multi-hop negative-feedback channel on top of directional evaporation, where messenger ants accelerate upstream adaptation when a failure significantly degrades a previously preferred direction.

Summary These two fault-tolerance mechanisms would improve AntNet's responsiveness in wireless multi-hop networks: beacon-driven directional evaporation provides fast local suppression of stale next-hop pheromone, and failure notification propagation disseminates negative feedback upstream to accelerate network-wide adaptation. These benefits come at the cost of additional control traffic (beacon and messenger ants), which introduces overhead and may contend with data traffic on the shared wireless medium. In Section 5, we will evaluate these add-on mechanisms, comparing their performance gains against their control overheads.

4 Implementation

We implement AntNet and the two fault-tolerance add-on mechanisms in ns-3, a discrete-event network simulator that provides detailed protocol-stack models (e.g. UDP/IPv4). Our routing protocol is implemented as a subclass of `Ipv4RoutingProtocol`, which is integrated in ns-3's IPv4 network layer and installed in every simulated node [9].

We implement `RouteOutput()` for locally generated packets and `RouteInput()` for received packets. When a node sends a locally generated packet, its `Ipv4L3Protocol` calls `RouteOutput()` to obtain a `Ipv4Route` entry specifying the next hop address and the outgoing interface. When a node receives a packet, its `Ipv4L3Protocol` calls `RouteInput()` to decide whether to forward, deliver locally, or drop the packet. These two hooks provide the integration points for our probabilistic next-hop selection and for scheduling and processing ant packets that update the routing state.

Our implementation consists of three components. First, each node maintains routing state including (i) a pheromone-based next-hop distribution (as a map of next-hop address and pheromone value) per destination, used to lookup `Ipv4Route` entries, and (ii) per-destination local traffic statistics required for pheromone updates. Second, we implement periodic control traffic using ns-3's event scheduler: forward ants (and, for the add-ons, beacon ants) are generated at configurable intervals and sent into the network as ordinary packets. Third, we define a lightweight `AntHeader` to encode control metadata, including the ant type, path memory and delay information stack carried by forward/backward ants, and (for failure notification) evaporation-related fields carried by messenger ants. To distinguish ant packets from normal data packets, we use the IPv4/6 protocol number 253, which is reserved for experimentation and testing purposes [10]. Upon reception, ant packets are dispatched by type (forward/backward/beacon/messenger) to perform the corresponding state updates or to trigger further ant generation or forwarding, and normal packets are forwarded, delivered, or dropped based on the searched `Ipv4Route` entry from the pheromone-based routing table.

We use ns-3's attribute system [11] to configure protocol parameters that may vary across

experiments, including the forward-ant interval I_a, beacon-ant interval I_b, beacon window size K, and the propagation threshold L. For the two add-on mechanisms, we additionally expose boolean flag attributes to enable or disable each mechanism independently, which facilitates ablation-style comparisons of effectiveness versus control overhead under identical simulation settings.

5 Evaluation

5.1 Experiment Setup

Figure 2 shows the evaluation topology used in our ns-3 experiments. We model an end-to-end communication scenario where a mobile source device sends packets to a mobile destination device over a heterogeneous path that traverses both wired and wireless multi-hop segments. On the left, a fixed wired backbone interconnects multiple routers via point-to-point links (solid lines) with the propagation delays annotated in the figure. This backbone is connected to a chain of wireless subnets through gateway nodes, forming a multi-hop wireless corridor between the two end hosts. Within each wireless subnet, nodes communicate over wireless links (dashed lines). The source and destination are placed in the edge wireless subnets and follow a mobility model during the simulation, so that link availability and quality can change over time. Every second, the source node would send 30 packets to the destination node with 210 bytes each. This setting allows us to evaluate how quickly AntNet and our fault-tolerance add-on mechanisms adapt to wireless dynamics while the end-to-end traffic still experiences realistic cross-domain routing over both wireless and wired components.

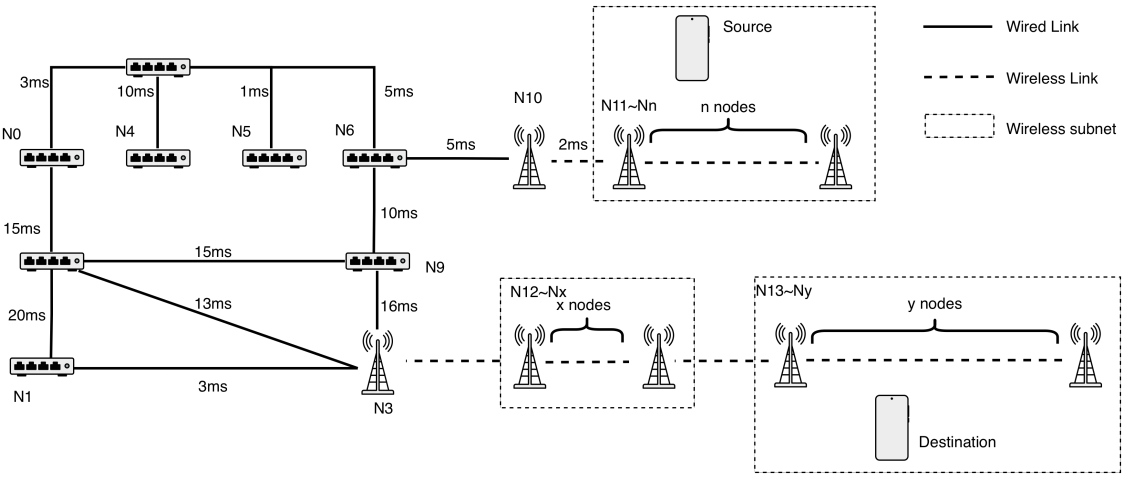
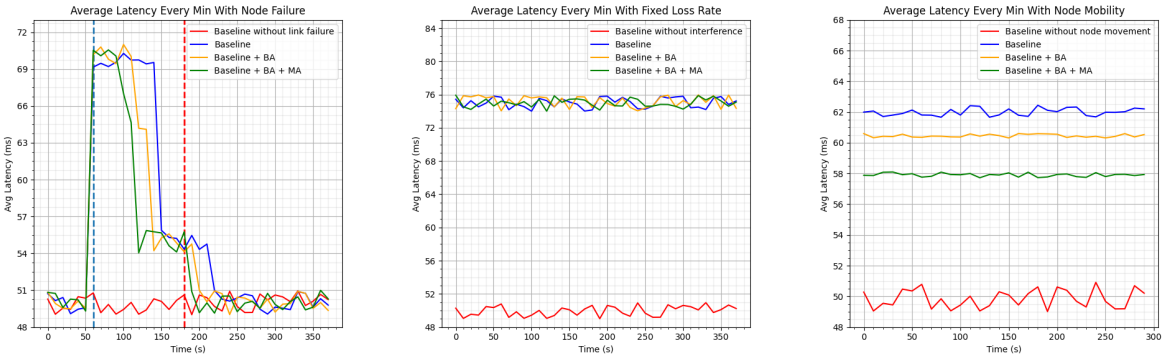


Figure 2: Topology in evaluation

We run experiment under three configurations: (i) baseline AntNet, (ii) baseline with beacon ants (BA) enabling neighbour monitoring and directional evaporation, and (iii) baseline with both beacon ants and messenger ants (BA+MA) enabling additional failure-notification propagation. In default, forward ants are scheduled in an interval of $I_a = 1$ second, beacon ants are scheduled in an interval of $I_b = 0.3$ seconds, beacon window size is $K = 5$, and notification threshold is $L = 60\%$. For beacon ants and messenger ants, we also do a sensitivity study on different values of beacon ant sending interval I_b, beacon window size K, and notification threshold L to test the trade-off between reflect-ability on failures and control packet overheads.



(a) Node failure (b) Random packet loss (c) Node Movement

Figure 3: Aggregated average latency over every minutes for different fault assumptions

5.2 Node/Link Failure

We first evaluate how quickly AntNet adapts to abrupt topology changes by injecting a controlled node failure on the current best path and then recover it back. In this experiment, as indicated in Figure 3(a), one intermediate node on the optimal route is forced down at $t = 60$s (shown in blue vertical line) and brought back up at $t = 180$s (shown in red vertical line). In addition to the three curves of three configurations (baseline, BA, and BA+MA), we also plot the vanilla AntNet's performance under no failure for reference.

As expected, the failure-free reference remains stable throughout the run. Once the failure occurs at $t = 60$s, all three AntNet-based configurations exhibit a latency spike, indicating that packets are temporarily routed along suboptimal detours while the routing state adapts. Baseline AntNet recovers the slowest, BA improves recovery speed by suppressing stale next-hop probabilities through beacon-driven directional evaporation, and BA+MA reacts the fastest by additionally propagating failure-induced negative feedback to upstream nodes. After the failed node recovers at $t = 180$s, latency drops back towards the pre-failure level, and the return to normal is faster than the initial reaction to failure. This asymmetry is consistent with AntNet's reinforcement mechanism: the protocol primarily updates routing state through successful end-to-end samples, so improvements (new good paths becoming available) are discovered and reinforced as soon as ants successfully traverse them, whereas degradations (broken directions) require explicit negative feedback or sustained sampling to eliminate stale preferences. Hence, Both add-on mechanisms reduce failure recovery time, with failure notification propagation providing the strongest improvement when the best path breaks, while preserving AntNet's ability to quickly re-exploit good paths after recovery.

5.3 Random Packet Loss

We then experiment with random packet loss to emulate signal interference. Specifically, instead of topology changes like node/link failures and recoveries, we apply a random packet drop with a fixed loss rate of 20% and measure the average latency over time. Figure 3(b) compares the same three protocol variants (baseline, BA, and BA+MA) against a no-interference reference.

In contrast to the node-failure experiment, all three AntNet-based configurations exhibit very similar latency under random packet loss. This outcome is expected since the packet losses affects both data packets and control packets (ants), reducing the reliability of the very signals used for

adaptation: forward/backward ants are more likely to be dropped before completing an end-to-end sampling cycle, and beacon/messenger ants can be lost before their information is incorporated by neighbours. Hence, under uncorrelated random loss that equally impacts control and data traffic, the bottleneck becomes observation reliability rather than routing responsiveness, and additional control mechanisms alone do not yield measurable latency gains.

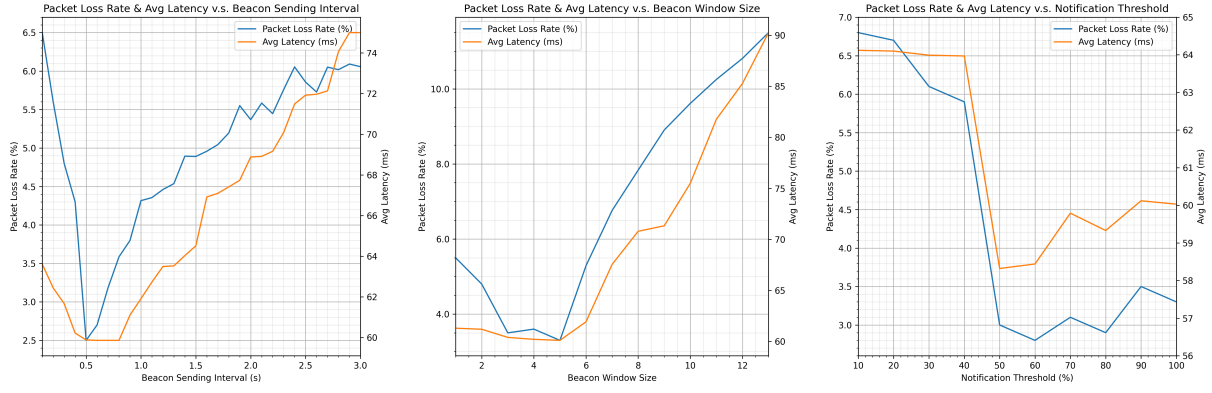
5.4 Node Mobility

We also conduct a mobility experiment to examine how the proposed fault-tolerance add-on affect end-to-end latency when link conditions continuously vary over time. The wireless infrastructure consists of a linear chain of base stations that remain within mutual coverage while being spaced 5 km apart. An end device (source) moves back and forth across the line at 80 km/h, repeatedly transitioning its best wireless attachment and inducing time-varying path quality and occasional route disruption.

Figure 3(c) reports the average end-to-end latency aggregated every minute over a 30-minute run. Baseline AntNet exhibits the highest latency, fluctuating around approximately 62 ms. Enabling beacon ants reduces the latency to roughly 60–61 ms, indicating that proactive neighbour monitoring and directional evaporation help suppress stale next-hop choices during mobility and shorten recovery time after transient link degradation. Adding messenger ants further reduces latency to around 58 ms and yields the lowest latency among the three settings. This suggests that propagating failure-related negative feedback beyond the detecting node accelerates upstream adaptation, thereby reducing the time during which packets are forwarded towards degraded or broken directions. Overall, this experiment provides evidence that both add-on mechanisms improve AntNet's adaptivity under mobility.

5.5 Sensitivity Study

We conduct a sensitivity study for the BA+MA configuration to understand how key parameters affect performance and control overhead. Figure 4 reports two metrics under identical traffic and topology settings: the end-to-end packet loss rate (left y-axis) and average latency (right y-axis). For the fault model, we include node mobility and three node failure/recovery events, as the previous experiments



(a) Beacon sending interval (b) Beacon window size (c) Notification Threshold

Figure 4: Packet loss rate and average latency for different add-on mechanism attributes

5.5.1 Beacon Sending Interval I_b

Figure 4(a) varies the beacon sending interval from $I_b = 0.1$s to $I_b = 3.0$s. When I_b is too small (very frequent beacons), both latency and loss rate increase, consistent with additional control traffic competing with data traffic on the shared wireless medium. When I_b is too large (infrequent beacons), the protocol reacts more slowly to neighbour changes and failures, which again increases latency and loss due to prolonged use of stale next-hop pheromones. Between these extremes, $I_b = 0.5$s appears to be a "sweet spot" where both packet loss and latency are minimized. This result also echos [6], which suggests two beacon ants per exploration interval (i.e., $I_b \approx I_a/2$). Overall, beacon ants should be frequent enough to detect sustained changes promptly, but not so frequent that it noticeably increases contention.

5.5.2 Beacon Window Size W

Figure 4(b) varies the beacon window size from $W = 1$ to $W = 13$ used to infer sustained beacon loss. A small-to-moderate window (between $W = 3$ to $W = 5$) achieves the best performance, while large windows lead to steadily increasing loss rate and latency. This trend aligns with the role of W as a stability-responsiveness knob: a larger W makes failure inference more conservative (reducing false positives), but delays detection and therefore delays directional evaporation and subsequent adaptation.

5.5.3 Notification Threshold L

Figure 4(c) varies the notification threshold from $L = 10\%$ to $L = 100\%$ (effectively disabling messenger-ant propagation) that decides whether a local evaporation event is "important" enough to trigger messenger-ant propagation. The optimal performance occurs around a moderate threshold, approximately $L = 50\%$. A low notification threshold increases messenger-ant traffic and may trigger excessive rebroadcasting under failures, increasing contention and packet loss, while a high threshold suppresses propagation and diminishes the benefit of upstream adaptation.

Summary Across all three parameters, the BA+MA mechanism demonstrates a consistent trade-off between faster reaction and additional control overhead. A moderate beacon interval, a small-to-moderate beacon window, and an intermediate notification threshold provide the best balance in our experiments, minimizing both packet loss and average latency.

6 Conclusion

In conclusion, this project improves the fault tolerance of an AntNet-style routing protocol in wireless multi-hop networks by integrating two complementary add-on mechanisms inspired by other ACO-style routing protocols [6] [7]: beacon-driven neighbour monitoring with directional pheromone evaporation (BA), and failure notification propagation via messenger ants (MA). We implemented baseline AntNet and these add-on mechanisms in ns-3 as a custom `Ipv4RoutingProtocol` with configurable attributes for ablation studies. Our experiments show that BA and MA reduce recovery latency under node mobility and node failures. Overall, the add-on mechanisms improve adaptivity at the cost of extra control overhead, and sensitivity results indicate that moderate parameter settings provide the best performance-overhead balance.

References

[1] M. Dorigo, M. Birattari, and T. Stutzle, "Ant colony optimization," *IEEE Computational Intelligence Magazine*, vol. 1, no. 4, pp. 28–39, 2006. DOI: 10.1109/MCI.2006.329691.

[2] G. Di Caro and M. Dorigo, "Antnet: Distributed stigmergetic control for communications networks," *Journal of Artificial Intelligence Research*, vol. 9, pp. 317–365, Dec. 1998. DOI: 10.5555/1622797.1622806.

[3] K. Jain, J. Padhye, V. N. Padmanabhan, and L. Qiu, "Impact of interference on multi-hop wireless network performance," in *Proceedings of the 9th Annual International Conference on Mobile Computing and Networking*, ser. MobiCom '03, San Diego, CA, USA: Association for Computing Machinery, 2003, pp. 66–80, ISBN: 1581137532. DOI: 10.1145/938985.938993. [Online]. Available: https://doi.org/10.1145/938985.938993.

[4] D. S. J. D. Couto, D. Aguayo, J. C. Bicket, and R. Morris, "A high-throughput path metric for multi-hop wireless routing," *Wirel. Networks*, vol. 11, no. 4, pp. 419–434, 2005. DOI: 10.1007/S11276-005-1766-Z. [Online]. Available: https://doi.org/10.1007/s11276-005-1766-z.

[5] ns-3 Project. "Ns3::ipv4routingprotocol class reference." ns-3 Documentation (Release 3.19), Doxygen API reference, Accessed: Dec. 17, 2025. [Online]. Available: https://www.nsnam.org/docs/release/3.19/doxygen/classns3_1_1_ipv4_routing_protocol.html.

[6] J. Grosso and A. Jhumka, "Fault-tolerant ant colony based-routing in many-to-many iot sensor networks," in *2021 IEEE 20th International Symposium on Network Computing and Applications (NCA)*, 2021, pp. 1–10. DOI: 10.1109/NCA53618.2021.9685935.

[7] G. Di Caro, F. Ducatelle, and L. M. Gambardella, "Anthocnet: An adaptive nature-inspired algorithm for routing in mobile ad hoc networks," *European Transactions on Telecommunications*, vol. 16, no. 5, pp. 443–455, 2005. DOI: 10.1002/ett.1062. [Online]. Available: https://doi.org/10.1002/ett.1062.

[8] P. Kao. "Introduction to the internet: Architecture and protocols." Section "Distance-Vector Protocols: Rule 5 Poisoning Expired Routes", Accessed: Dec. 17, 2025. [Online]. Available: https://textbook.cs168.io/routing/distance-vector.html#rule-5-poisoning-expired-routes.

[9] ns-3 Project. "16.4. routing overview." Models documentation, Accessed: Dec. 17, 2025. [Online]. Available: https://www.nsnam.org/docs/models/html/routing-overview.html.

[10] B. E. Carpenter and S. Jiang, *Transmission and processing of ipv6 extension headers*, Request for Comments: 7045, Dec. 2013. DOI: 10.17487/RFC7045. [Online]. Available: https://www.rfc-editor.org/info/rfc7045.

[11] ns-3 Project. "2.4. configuration and attributes." Manual documentation, Accessed: Dec. 17, 2025. [Online]. Available: https://www.nsnam.org/docs/manual/html/attributes.html.

A Mathematics in AntNet

The following subsections describe the mathematical details of exploration and reinforcement in AntNet [2].

A.1 Routing State

A.1.1 Pheromone Routing Table

For each destination d, the pheromone routing table maintains a probability distribution over possible next hops. Concretely, the entry for d is a map from neighbour n to a pheromone value $p_{(d,n)}$, denoted as $\langle n, p_{(d,n)} \rangle$. For each destination d, pheromone values satisfy

$$\sum_{n \in N} p_{(d,n)} = 1$$

where N is the set of current neighbours.

A.1.2 Local Traffic Statistic Table

For each destination d, the local traffic statistics table stores the estimated mean delay μ_d and variance σ_d^2 of the end-to-end delay, a sliding window of size K_{traffic} containing the most recent K_{traffic} delay samples, and a historical data flow amount f_d. The use of these values is described below.

A.2 Forward Ant Destination

Each node sends a forward ant to a destination selected according to historical traffic demand. The traffic amount f_d (e.g., number of packets or bytes originated locally and delivered to d) is recorded in the local traffic statistics table. The destination selection probability is:

$$\Pr(\text{destination} = d) = \frac{f_d}{\sum_{i \in \text{all possible destination nodes}} f_i}$$

Hence, the exploration of forward ants would follow the pattern of data traffic distribution. If $\sum_i f_i = 0$, which would happens at the beginning of the exploration where there is no data sent out, the destination is selected uniformly at random among all possible destinations.

A.3 Next-Hop Finding

When sending a locally generated packet or forwarding a received packet, the next hop is selected probabilistically based on pheromone values. In addition to historical recorded from routing state, the performance of the next hop n is also affected by real time congestion in the link between the current node and next hop n, represented by the queue length l_n of the link. Hence, a heuristic term is defined as:

$$p'_{(d,n)} = 1 - \frac{l_n}{\sum_{i \in N} l_i}$$

The pheromone values $p_{(d,n)}$ for each possible next hop n for that destination d is adjusted by the heuristic correction p'_n:

$$\Pr(\text{next hop} = n) = \frac{p_{(d,n)} + \alpha p'_{(d,n)}}{1 + \alpha(|N| - 1)}$$

where α controls the weight of the heuristic correction.

A.4 Routing State Update

When a node m receives a backward ant, it updates routing state using the observed end-to-end delay $t_{m \to d}$ carried by the ant to destination d. The local traffic statistics are updated by exponential smoothing:

$$\mu_d \leftarrow \mu_d + \eta(t_{m \to d} - \mu_d)$$
$$\sigma_d^2 \leftarrow \sigma_d^2 + \eta((t_{m \to d} - \mu_d)^2 - \sigma_d^2)$$

where η is a smoothing factor. The observation window is also updated with $t_{m \to d}$, and $t_{m \to d}^{\text{best}}$ denotes the best observed delay within the current window.

Let n be the next hop on the sampled path, which is the previous node along the backward ant's path. To update the pheromone $p_{(d,n)}$ for the sampled destination d and next hop n, the reward $r_{(d,n)}$ is calculated as:

$$r_{(d,n)} = c_1 \frac{t_{m \to d}^{\text{best}}}{t_{m \to d}} + c_2 \frac{(\mu_d + c_3 \frac{\sigma_d}{\sqrt{K_{\text{traffic}}}}) - t_{m \to d}^{\text{best}}}{((\mu_d + c_3 \frac{\sigma_d}{\sqrt{K_{\text{traffic}}}}) - t_{m \to d}^{\text{best}}) + (t_{m \to d} - t_{m \to d}^{\text{best}})}$$

where c_1, c_2, and c_3 are constants. The pheromone value for the sampled next hop is updated by:

$$p_{(d,n)} \leftarrow p_{(d,n)} + r_{(d,n)} \times (1 - p_{(d,n)})$$

The pheromone value for other next hops for destination d is normalized to keep the sum to one:

$$p_{(d,n')} \leftarrow p_{(d,n')} - r \times p_{(d,n')} \text{ for each possible next hop } n' \neq n$$